Evaluating Phishing Email Detection Against Adversarial Attacks: Using Traditional, Deep Learning and Transformer Models

Wing Lee
CS 271 Top Machine Learning
wing.lee@sjsu.edu

Ansh Vaghani
CS 271 Top Machine Learning
anshmukeshbhai.vaghani@sjsu.edu

***Abstract*—In the modern world phishing emails is a problem that persists as a cyber security threat, were it exploits human trusts to steal sensitive credentials and personal information. Machine learning models have improved over the years with a high detection accuracy on static datasets, but they are rarely tested against adversarial manipulated emails that is similar to real world attacker techniques. In this report, we present a deep comparative model of six classifiers going from traditional machine learning, deep learning and also transformers. Naive Bayes, Logisic Regression, Support Vector Machine (SVM), Multilayer Perceptron (MLP), Convolutional Neural Network (CNN), and DistilBERT. Each model is tested under three different conditions, first is clean emails, character substitution attacks and then a combined adversarial attack of Homoglyphs and invisible character injection. Our results show that standard models suffer a lot in performance under attack, while adversarially trained models and transformer model maintain higher performance. On this project we report Accuracy, F1-Score, Area Under the ROC Curve (AUC), and the Equal Error Rate (EER) across all its conditions.**

***Index Terms*—phishing detection, adversarial attacks, homoglyphs, machine learning, deep learning, DistilBERT, MLP, CNN, SVM, phishing email**

I. Introduction

Phishing emails are a major cybersecurity problem. Attackers trick people into giving up passwords, financial details, and personal information by making emails look legitimate. Machine learning has helped a lot with detecting these emails, but most models are only tested on clean, unmodified datasets. In the real world, attackers do not send clean emails — they modify them using tricks like swapping characters ("paypal" becomes "paypa1"), using look-alike Unicode characters, and adding fake legitimate-sounding text to confuse filters [1].

Most research in this area follows the same pattern: train a model, test it on a static dataset, report high accuracy, and call it done. Ripa et al. [7] tested Naive Bayes and got 95.15% accuracy on phishing emails, but only on unmodified data. Bagui et al. [10] compared Naive Bayes, SVM, Decision Tree, LSTM, and CNN and found deep learning did slightly better, but again, only on clean emails. Ishwarya et al. [9] ran Naive Bayes, SVM, KNN, and Random Forest and hit 98.9% accuracy, with no testing on modified emails at all. Form et al. [8] built a hybrid feature system using keywords, URLs, and sender behavior, reaching 97.25% with SVM, but keyword-based systems break the moment an attacker substitutes one character. Abahussain and Harrath [3] used regular expressions and domain databases to catch malicious emails, which fails the same way — change one character in a token and the regex no longer matches. Ayodele et al. [5] proposed MLAPT, a behavioral pattern matching framework built on human-labeled data, but it assumed phishing emails follow consistent patterns. Helmi et al. [6] focused on detecting phishing links in email attachments using a decision tree, which does not help when the attack is in the email body itself. Joseph and Srinivasan [2] proposed an adaptive AI system with NLP-based real-time analysis, but tested it against traditional phishing types, not character-level manipulation. Gülserliler et al. [4] fine-tuned BERT, DistilBERT, and ELECTRA for Turkish phishing emails and got good results, but never tested adversarial robustness. Chien and Khethavath [1] are one of the few papers to actually point out that character substitutions like "paypa1" are a real gap, and that models trained on prefetched datasets do not generalize well to real emails.

The gap across all of this work is the same: nobody actually tests what happens to these models when the emails are manipulated. That is what this paper does. We train six models — Naive Bayes, Logistic Regression, MLP, SVM, CNN, and DistilBERT — on the Nazario and spam-ham datasets [11] [12] and test them under three conditions: (1) clean emails, (2) character-substituted emails using ASCII swaps like 'a' $\rightarrow$ '@' and 'o' $\rightarrow$ '0', and (3) a harder adversarial attack that combines Unicode homoglyph substitution, zero-width character injection, and ham phrase injection. We also test whether training on augmented data that includes manipulated emails makes models more robust.

Two main findings came out of this. First, robust training works well against character substitution — SVM went from 46.2% to 98.4% accuracy just by training on augmented data. Second, when we hit all models with the harder adversarial attack, every single one collapsed to 47–54% accuracy, including DistilBERT. Interestingly, Logistic Regression with robust training got the highest AUC of 0.9289 under the adversarial attack, beating DistilBERT at 0.9065, which tells us that a more complex model does not always mean a more robust one.

II. Related Work

A lot of previous publications has been done on phishing email detection over the past few years. Most of them uses standard machine learning classifiers on clean datasets.

Ayodele et al. [5] built MLAPT, one of the earlier machine learning frameworks for phishing prevention. It used behavioral patterns from human-labeled phishing scenarios to flag suspicious emails. The problem is it relied on predefined features and assumed phishing emails would follow predictable patterns, which is not really true anymore.

Abahussain and Harrath [3] took a different approach using regular expressions and a domain database to inspect email URLs and sender addresses. It worked for known phishing patterns but falls apart when an attacker changes even one character in a URL or keyword, since the regex no longer matches.

Helmi et al. [6] built an application that uses a decision tree to find phishing websites embedded in email attachments. It works specifically for link-based attacks and does not help when the attack is in the email text itself.

Form et al. [8] combined content, URL, and behavioral features into a hybrid system and got 97.25% accuracy with SVM on 500 phishing and 500 legitimate emails. The issue is that blacklisted keywords and URL patterns are easy to get around with character substitution.

Ripa et al. [7] looked at phishing detection across three attack types — URLs, emails, and websites — using multiple classifiers. For email detection they used Naive Bayes and got 95.15% accuracy. Good numbers, but all on clean unmodified data.

Ishwarya et al. [9] ran Naive Bayes, SVM, KNN, and Random Forest on a Kaggle phishing dataset with standard preprocessing and hit 98.9% accuracy. Same story — no adversarial testing.

Bagui et al. [10] compared a wider set of models including LSTM and CNN alongside traditional classifiers, finding that deep learning models did slightly better at picking up contextual patterns in email text. Still, only tested on clean data.

Chien and Khethavath [1] went deeper than most, analyzing multiple email features and pointing out that character substitutions like "paypa1" are a real attack vector that most models are not built to handle. They also showed that models trained on prefetched datasets do not always perform well on real emails. This directly motivated our work.

Joseph and Srinivasan [2] proposed an adaptive AI system using NLP and both supervised and unsupervised learning for real-time phishing detection. It improved false positive rates over traditional systems, but the evaluation did not include any character-level adversarial attacks.

Gülserliler et al. [4] fine-tuned BERT, DistilBERT, and ELECTRA specifically for Turkish phishing emails and showed that language-specific models work better than general ones for non-English content. Useful finding, but adversarial robustness was not tested at all.

The common thread across all of this is that nobody tests what happens when the emails are actually manipulated. High accuracy on clean data does not tell you how a model holds up when an attacker knows what they are doing. That is the gap this paper fills.

III. Methodology

A. Datasets

Two public datasets were used for this project obtained from Kaggle.

- **Nazario Phishing Dataset**: A collection of real-world phishing emails sourced from documented phishing campaigns, containing email body text with binary labels (phishing emails = 1, legit emails = 0).
- **Spam/Ham Dataset**: A benchmark of both spam and ham email classification dataset containing labeled email text.

We first merge the two datasets together and removing any null entries. The combined dataset of both Nazario and Spam/Ham dataset contains **6,736 emails**: 3,672 legit (label 0) and 3,064 phishing/spam (label 1). We then used an 80/20 stratified training test split, giving 5,388 training samples and 1,348 testing samples.

B. Preprocessing

The following shown in Figure 1, is how all email texts went following this pipeline:

1) Lowercasing
2) HTML tag removal via regex
3) URL replacement with the token `url`
4) Numeric replacement with the token `num`
5) Punctuation removal
6) Whitespace normalization

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'<[^>]+>', ' ', text)
    text = re.sub(r'http\S+|www\.\S+', ' url ', text)
    text = re.sub(r'\d+', ' num ', text)
    text = re.sub(r'[^\w\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

Fig. 1. Text preprocessing pipeline applied to all email bodies.

C. Text Represenation

The models in this project used different input representations depending on their model.

TF-IDF Vectorization: This was used to apply Naive Bayes, Logisitic Regression, and SVM. The TF-IDF vectorizer has 5,000 maximum features, unigram and bigram (n-gram range (1,2)), and sublinear TF scaling were used. Standard Naive Bayes and Logistic Regression and the vectorizer was fitted on the clean training only. Robust Naive Bayes, Robust Logistic Regression, MLP, and SVM used a TF-IDF fitted on the training set of clean and character substituted emails.

Tokenization/Padding: Used for CNN. Keras tokenizer was used with a vocabular of 10,000 tokens and an out of vocabulary `<OOV>` token was also added into the training set. Each email then was converted into a sequence of integer token IDs, then either padded or truncated into a fixed length of 200 tokens.

DistilBERT Tokenization: Pre-trained `distilbert based uncased` it is a tokenizer that encodes email texts into token IDs with sequence padding and truncated to a max length of 256 tokens. The transformer then processes the tokens through its layers to produce contextual representations of the email text.

D. Adversarial Attack Design

Three evaluation conditions were applied to the test set:

(1) Clean: Unmodified preprocessed email text.

(2) Character Substitution (Homoglyph-ASCII): ASCII characters are replaced with visually similar alternatives using the mapping: {`a:@, o:0, i:1, e:3, s:$, l:1, g:9, t:7`}. This simulates obfuscation techniques commonly used by attackers to bypass keyword based filters.

(3) Combined Adversarial Attack: A more sophisticated evasion strategy combining:

- *Unicode homoglyph substitution*: Latin characters replaced with visually identical Cyrillic characters (e.g., Latin "a" → Cyrillic "a" (U+0430)).
- *Invisible character injection*: Zero-width spaces (U+200B) and zero-width non-joiners (U+200C) inserted between characters to disrupt tokenization.
- *Ham-phrase injection*: Legitimate-sounding phrases (e.g., "privacy policy", "terms of service") inserted into phishing emails to shift the model's prediction toward legitimate.

E. Adversarial Training

For Naive Bayes, Logistic Regression, and MLP, robust versions were trained on the dataset combining the original training emails with their character substituted counterparts, we doubled the training set to 10,776 samples. This adversarial training strategy is designed to expose models to manipulated inputs during its training, help improve generalization under attacks.

F. Models

1) Naive Bayes: A Multinomial Naive Bayes classifier trained on TF-IDF features. Trained on a single analytical pass with no epoch based optimization.

2) Logistic Regression: A logistic regression classifier with L2 regularization (`max_iter=1000`). Trained on TF-IDF features.

3) Support Vector Machine (SVM): A `LinearSVC` with `C=1.0` and 2,000 maximum iterations, wrapped in a `CalibratedClassifierCV` with 5-fold cross-validation to enable probability output for ROC and EER computation. This was Trained on robust TF-IDF features.

4) Multilayer Perceptron (MLP): A three-layer feedforward neural network with architecture: Dense(128, ReLU), Dropout(0.3), Dense(64, ReLU), Dropout(0.3), Dense(32, ReLU), Dense(1, Sigmoid). Optimized with Adam and binary cross-entropy loss. Trained on TF-IDF robust features with EarlyStopping (patience=5) monitoring validation loss over up to 50 epochs.

5) Convolutional Neural Network (CNN): A 1D TextCNN with architecture: Embedding(10000, 128), Conv1D(128 filters, kernel=5, ReLU), GlobalMaxPooling1D, Dense(128, ReLU), Dropout(0.4), Dense(64, ReLU), Dropout(0.3), Dense(1, Sigmoid). Trained on sequence-encoded robust data with EarlyStopping (patience=5).

G. DistilBERT

This was trained on 5 epochs with a linear learning rate, using AdamW optimizer and batch size of 16. This training was performed on Google Colab using NVIDIA T4 GPU.

H. Evaluation Metrics

Models are evaluated using:

- **Accuracy**: Proportion of correct predictions.
- **F1-Score**: Harmonic mean of precision and recall.
- **AUC**: Area under the Receiver Operating Characteristic curve, measuring discriminative ability across all thresholds.
- **EER**: Equal Error Rate, the point where False Positive Rate equals False Negative Rate. Lower EER indicates better security-relevant robustness.

IV. Experiments and Results

A. Experimental Setup

All experiments used a fixed random seed (42) with a stratified 80/20 train/ test split. Naive Bayes, Logistic Regression, and SVM were implemented using scikit-learn. MLP and CNN were built using TensorFlow/Keras. DistilBERT was implemented using HuggingFace Transformers with PyTorch.

B. Clean Email Performance

Table I reports performance on unmodified test emails. All models achieve strong baseline accuracy, with DistilBERT achieving the highest accuracy (99.26%) and lowest EER (0.0068). The MLP and SVM perform comparably at approximately 97–98% accuracy.

TABLE I
PERFORMANCE ON CLEAN EMAILS

Model	Accuracy	F1	AUC	EER
Naive Bayes	0.9369	0.9317	0.9842	0.0571
Logistic Regression	0.9718	0.9695	0.9970	0.0259
Naive Bayes (Robust)	0.9362	0.9304	0.9829	0.0626
LR (Robust)	0.9681	0.9651	0.9959	0.0340
MLP	0.9740	0.9713	0.9980	0.0231
SVM	0.9763	0.9739	0.9979	0.0231
CNN	0.9785	0.9764	0.9976	0.0218
SVM (Normal)	0.9822	0.9805	0.9987	0.0177
DistilBERT	0.9926	0.9918	0.9996	0.0068

C. Character Substitution Attack

Table II reports performance when phishing test emails are subjected to ASCII homoglyph substitution. Standard Naive Bayes and Logistic Regression goes down significantly to (64.2% and 68.1% accuracy), confirming their dependence on exact token matching. Adversarially trained variants recover substantially. MLP, SVM, CNN, and DistilBERT all got above 96% accuracy due to robust training.

TABLE II
PERFORMANCE UNDER CHARACTER SUBSTITUTION ATTACK

Model	Accuracy	F1	AUC	EER
Naive Bayes	0.6417	0.6430	0.7330	0.3456
Logistic Regression	0.6813	0.6813	0.7998	0.2585
Naive Bayes (Robust)	0.9318	0.9277	0.9829	0.0680
LR (Robust)	0.9770	0.9750	0.9972	0.0272
MLP	0.9696	0.9664	0.9974	0.0259
SVM	0.9844	0.9829	0.9984	0.0150
CNN	0.9748	0.9726	0.9967	0.0245
SVM (Normal)	0.4622	0.6257	0.7481	0.3306
DistilBERT	0.9644	0.9605	0.9943	0.0354

D. Combined Adversarial Attack

Table III reports performance under the combined adversarial attack. All models experience notable down performance, this shows that more sophisticated evasion strategies are strong even against robustly trained classifiers. DistilBERT achieves the highest AUC (0.9065) and EER (0.1864), while Logistic Regression (Robust) achieves the best accuracy (0.6422) among non-transformer models.

TABLE III
PERFORMANCE UNDER COMBINED ADVERSARIAL ATTACK

Model	Accuracy	F1	AUC	EER
Naive Bayes (Robust)	0.5423	0.6652	0.8920	0.1918
LR (Robust)	0.6422	0.6422	0.9289	0.1524
MLP	0.4755	0.6339	0.7527	0.3007
SVM	0.4718	0.6322	0.8624	0.2136
CNN	0.5237	0.6563	0.8032	0.2830
SVM (Normal)	0.4629	0.6287	0.8811	0.1986
DistilBERT	0.5408	0.6645	0.9065	0.1864

E. ROC Curve Analysis

Figure 2 presents the ROC curves for all models across the three evaluation conditions. On clean emails (left plot), all models got almost perfect separation with AUC values at or near 1.00. Under character substitution (middle plot), standard Naive Bayes and Logistic Regression curves collapse toward the diagonal (AUC 0.73 and 0.80), while all robust trained models maintain a high AUC. The combined adversarial attack (right plot) shows the most significant change, with DistilBERT and Logistic Regression (Robust) maintaining the highest AUC values (0.91 and 0.93).

F. AUC Comparison

Figure 3 shows the AUC across all three conditions per each model. The contrast between standard and robust model is shown in the character substitution condition, where standard Naive Bayes and LR drop significantly while their robust models remain stable.

G. EER Comparison

Figure 4 shows EER across all model conditions. Lower EER means a better result. Under the combined adversarial attack, all models show a higher EER, but DistilBERT and Logistic Regression (Robust) maintain the lowest EER values, proving that they are superior to performance relevant to security.

H. Accuracy Comparison

Figure 5 summarizes accuracy across all models conditions. Under character substitution, standard models drop to 46 to 68% while robust versions stayed 93 to 98%. Under the combined adversarial attack, all models drop to 47 to 64%, this shows the challenge with doing multi strategy evasion.

I. Training Convergence

Figures 6, 7, and 8 show the training and validation accuracy and loss curves for MLP, CNN, and DistilBERT. All three models converge smoothly. MLP and CNN both show train/validation curves with minimal overfitting, stopping at epoch 7 due to the EarlyStopping method. DistilBERT shows a slight increase in validation loss after epoch 3, suggesting minor overfitting, though validation accuracy remains stable at approximately 97 to 98%.

V. DISCUSSION

A. Impact of Adversarial Training

The most interesting finding is how well adversarial training did for traditional models. Standard Naive Bayes and Logistic Regression drop to 64% and 68% accuracy under character substitution, while their robust models went to 93% and 97%. This shows that manipulated examples during training help improves TF-IDF-based model robustness without requiring architectural changes.

B. Transformer Robustness

DistilBERT achieves the highest clean accuracy (99.26%) and has a high performance under character substitution (96.44%) without any adversarial training. This is because DistilBERT understands the meaning of the words, rather than relying on exact keyword matches, making it harder to fool with character substitutions.

C. Combined Adversarial Attack Limitations

Under the combined adversarial attack incorporating Homoglyphs and invisible characters, all models went down significantly to 46 to 64% accuracy. This proves that multi-strategy evasion poses a fundamentally harder challenge. Also Homoglyph substitution specifically defeats tokenization-based approaches since Cyrillic lookalike characters are treated as entirely distinct tokens or new characters.

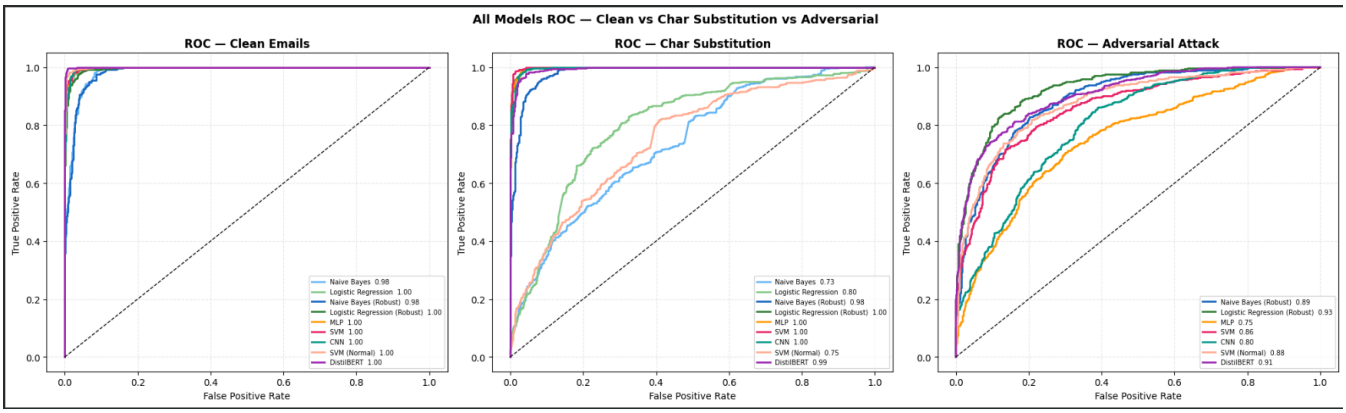

Fig. 2. ROC curves for all models under Clean, Character Substitution, and Combined Adversarial Attack conditions.

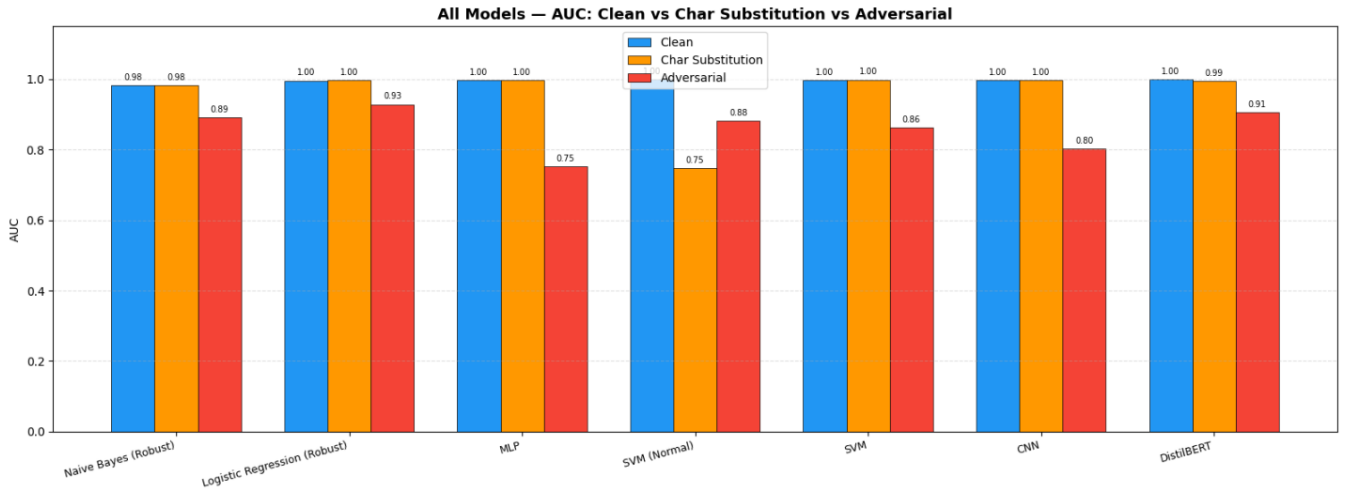

Fig. 3. AUC comparison across all models under three evaluation conditions.

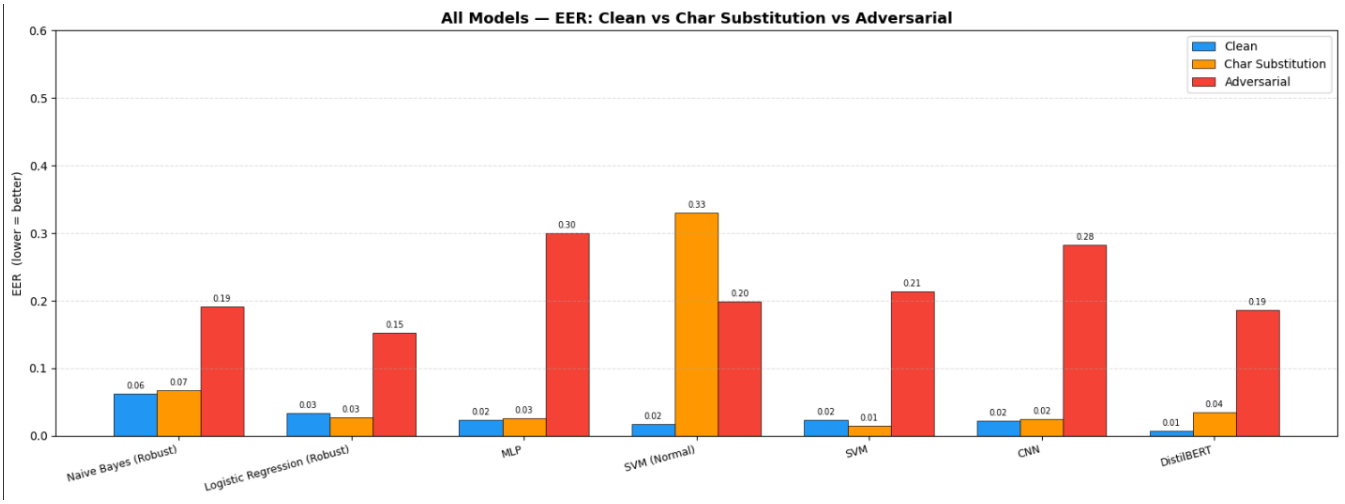

Fig. 4. EER comparison across all models. Lower values indicate better robustness.

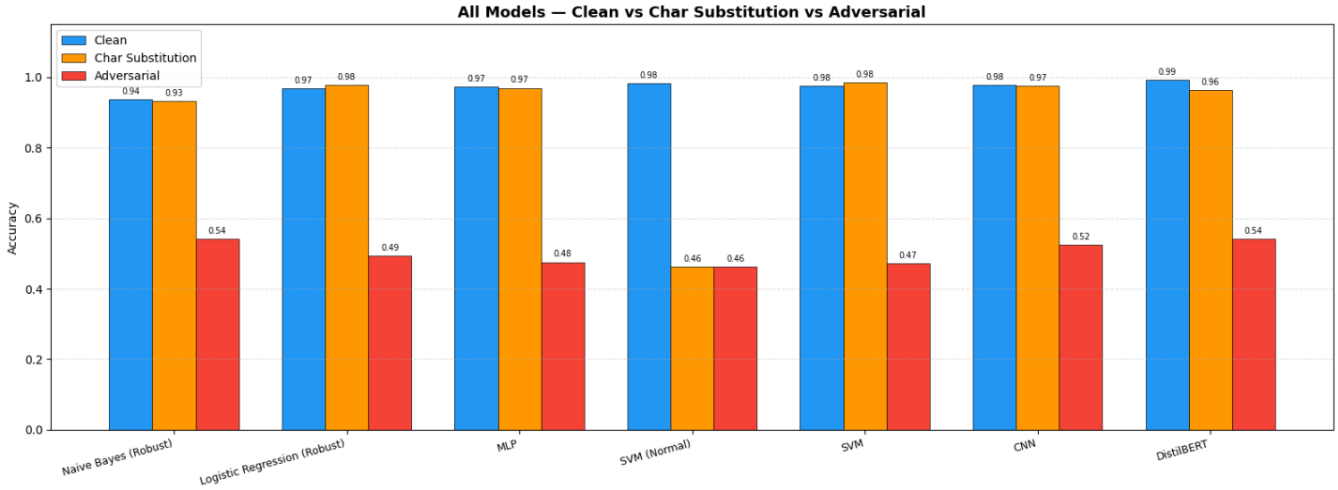

Fig. 5. Accuracy comparison across all models under three evaluation conditions.

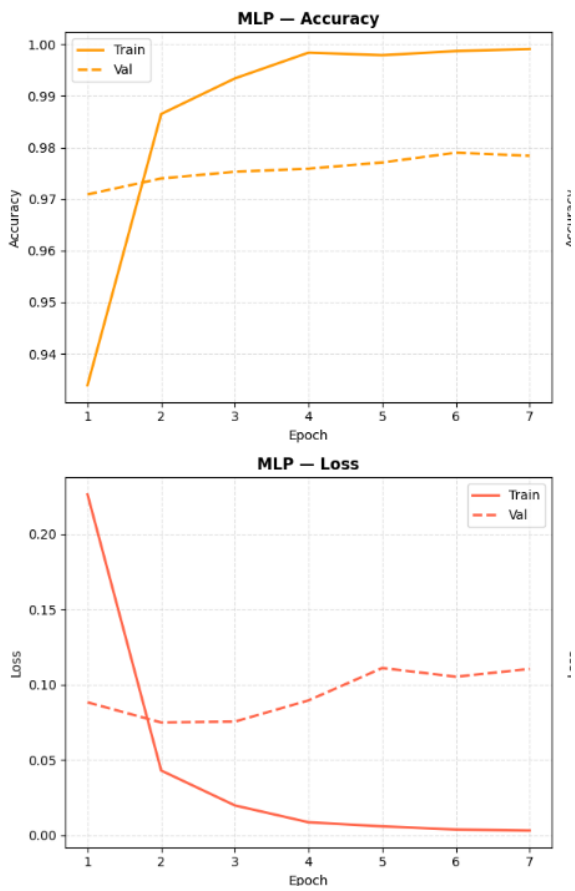

Fig. 6. MLP training and validation accuracy/loss curves.

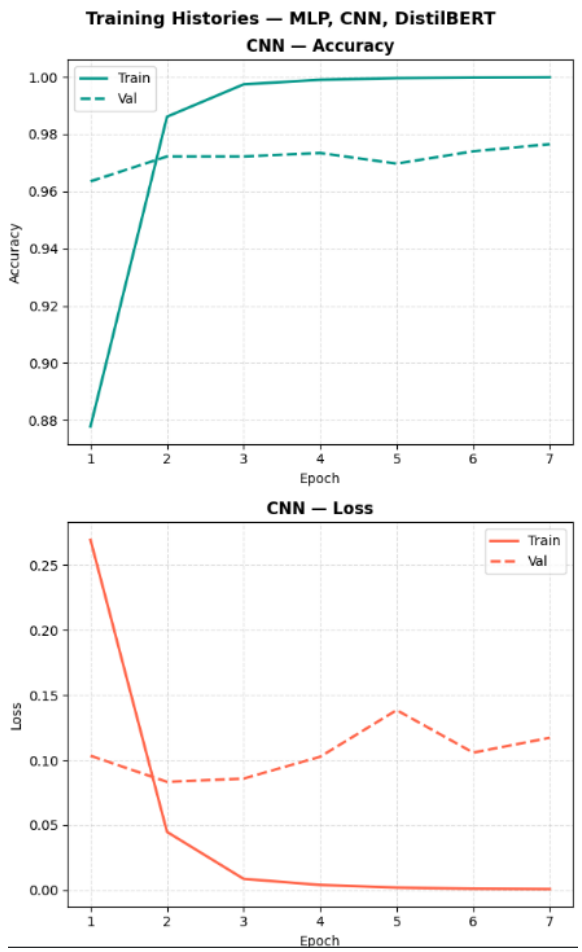

Fig. 7. CNN training and validation accuracy/loss curves.

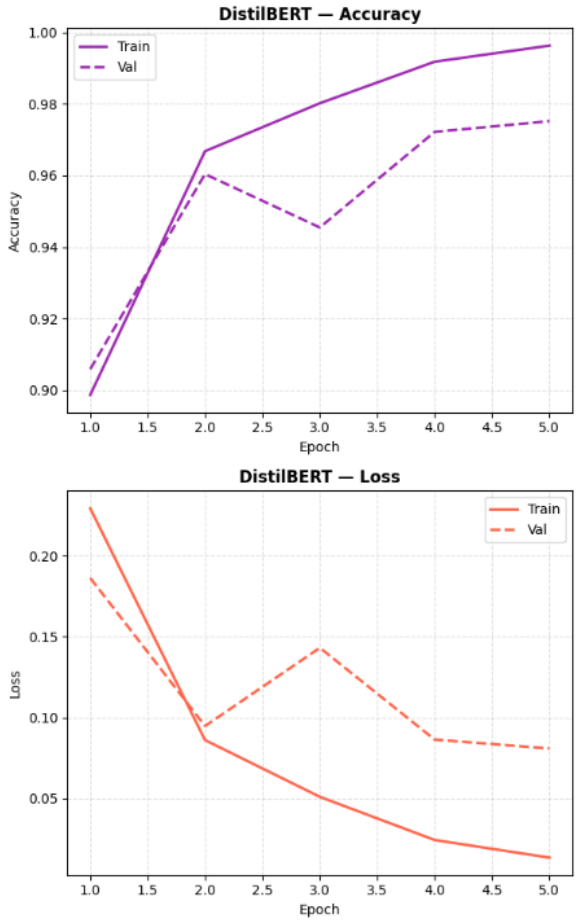

Fig. 8. DistilBERT training and validation accuracy/loss curves over 5 epochs.

D. SVM Behavior

The standard SVM (Normal) got the highest clean accuracy among traditional models (98.22%) but goes down entirely under character substitution (46.22%), worse than Naive Bayes. This is because the LinearSVC's large-margin decision boundary, learned entirely from clean tokens, provides no generalization to substituted variants. The robustly trained SVM however maintains 98.44% accuracy under character substitution and achieves the lowest EER (0.0150) in that condition.

VI. Conclusion

This paper presented a detailed evaluation of six phishing email detection models across three adversarial attack conditions. Standard models trained on clean data suffer severe accuracy down performance of up to 53% under Homoglyph-based character substitution attacks. Adversarial training helped bring robustness for traditional models, with Robust LR obtaining a 97.7% accuracy under character substitution. DistilBERT is robust to character substitution without adversarial training due to its understanding of email content. All models go down significantly under combined adversarial attacks, indicating that multiple evasion methods remains an open challenge. EER gives a good insight of security oriented metric.

VII. Future Work

Our future work would include trying out a more modern database or even a larger database so that we can run more training and testing to see how well each of the six models perform. Another would be attempting adversarial training with Homoglyphs and invisible characters injection to help improve the model against a more modern evasion technique. Trying out different models like LSTM would also see how well they would do compared to the six models we current have. As well as testing these models on actual real emails instead of datasets since those will always become outdated eventually.

References

[1] A. Chien and P. Khethavath, "Email Feature Classification and Analysis of Phishing Email Detection Using Machine Learning Techniques," in *Proc. IEEE CSDE*, Dec. 2023, doi: 10.1109/csde59766.2023.10487729.

[2] A. S. K. Joseph and S. Srinivasan, "Anti-Phishing Adaptive AI Systems: Efficiently Countering Social Engineering Attacks by Real-Time Analysis of Email Content," in *Proc. ICCIES*, Apr. 2025, pp. 1–6, doi: 10.1109/iccies63851.2025.11032758.

[3] O. Abahussain and Y. Harrath, "Detection of Malicious Emails through Regular Expressions and Databases," in *IEEE Xplore*, Sep. 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8910291

[4] E. Gülserliler, D. E. Keneş, and Ş. Bahtiyar, "Turkish Email Phishing Detection Using Fine Tuned Large Language Models," in *Proc. ACIT*, Sep. 2025, pp. 487–490, doi: 10.1109/acit65614.2025.11185591.

[5] T. Ayodele, C. A. Shoniregun, and G. Akmayeva, "Anti-phishing Prevention Measure for Email Systems," in *IEEE Xplore*, Jun. 2012. [Online]. Available: https://ieeexplore.ieee.org/document/6280179

[6] R. A. A. Helmi, C. S. Ren, A. Jamal, and M. I. Abdullah, "Email Anti-Phishing Detection Application," in *IEEE Xplore*, Oct. 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8906316

[7] S. P. Ripa, F. Islam, and M. Arifuzzaman, "The Emergence Threat of Phishing Attack and The Detection Techniques Using Machine Learning Models," in *Proc. ACMI*, Jul. 2021, doi: 10.1109/acmi53878.2021.9528204.

[8] L. M. Form, K. L. Chiew, S. N. Sze, and W. K. Tiong, "Phishing Email Detection Technique by Using Hybrid Features," in *Proc. IEEE*, 2016, doi: 10.1109/iiai-aai.2016.68.

[9] Ishwarya R, Muthumani S, Siva, and Suriya S, "Separation of Phishing Emails Using Probabilistic Classifiers," in *Proc. ICACCS*, Mar. 2023, pp. 1676–1679, doi: 10.1109/icaccs57279.2023.10112826.

[10] S. Bagui, D. Nandi, S. Bagui, and R. J. White, "Classifying Phishing Email Using Machine Learning and Deep Learning," in *IEEE Xplore*, Jun. 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8885143

[11] N. Abdullah Alam, "Phishing Email Dataset (Nazario)," *Kaggle*, 2023. [Online]. Available: https://www.kaggle.com/datasets/naserabdullahalam/phishing-email-dataset

[12] Venky73, "Spam Mails Dataset," *Kaggle*, 2021. [Online]. Available: https://www.kaggle.com/datasets/venky73/spam-mails-dataset